

PACT: a reproducible systems evaluation

Noah Dylan Pelegrini · Independent Researcher
contact@11adyy.dev
<https://github.com/11adyy/pact-research>

Measured 2026-10-02 · seed 20261002 · offline deterministic mode

Abstract

We evaluate the supplied, unchanged PACT runtime using controlled services and exact task oracles. The recorded suite contains **23,382 timed or behavioral executions**, including 9,600 healthy runs and 12,800 fault/policy runs. Full PACT produces the exact expected output in 1,200/1,200 healthy executions. Experiments measure orchestration cost, declared policy enforcement, failure containment, retries, scaling, traces, and binding substitution. Negative probes expose output-validation gaps and effects after timeout. These measurements support bounded systems claims; they do not measure LLM reasoning quality or inference savings.

1. Experimental design

Component	Recorded protocol
Tasks	Sum; maximum; constrained weighted selection; tagged-text aggregation
Healthy cases	100 distinct generated inputs per task; 3 repetitions; 8 arms
Robustness	16 scenarios; 100 distinct inputs per scenario; 8 arms
Additional probes	30 invalid-structure runs; 160 substitution executions; 600 chain runs; 192 fan-out runs
Ground truth	Independent exact oracle; oracle-only fields excluded from service/model input
Measurement	perf_counter_ns; warmup excluded; seeded randomized healthy/scaling order
Statistics	Case-cluster bootstrap, 2000 draws; paired per-case means; Wilson case intervals

Repeated runs do not increase the independent-case count. The four families are controlled, synthetic workloads, with no held-out real-world benchmark. Trials are serial except declared internal fan-out. Services record an isolated in-memory effect ledger; no production actions occur. Audit persistence is disabled and excluded from the measured cost.

Environment: Python 3.12.14; Linux-6.18.44-x86_64-with-glibc2.39; 9 reported CPUs. Shared host, without CPU pinning or control over other tenants. Regression validation briefly overlapped the start of the healthy run; randomized comparisons and intervals remain descriptive of this session, not hardware-independent performance guarantees.

2. Baselines and healthy correctness

Arm	Purpose
direct	Same four functions called directly; orchestration lower bound
transport	Same functions via the native PythonCallInvoker and per-call threads
manual_guarded	Transport plus independent policy/type checks and the same integrity gate
pact	Actual engine, scheduler, mappings, capability contracts and binding resolution
Ablations	Remove policy metadata; bypass capability checks; trace flag off; freeze bindings

The primary comparator is manual_guarded: both arms make four workload calls plus one integrity-gate call. Its type checks are stricter than the current runtime. Transport/direct omit the governance gate, so their timings are supplemental lower bounds. Initialization is outside trial timing; recorded construction costs are single observations, without confidence intervals.

Task	PACT exact cases	PACT mean ms	Manual mean ms	Paired delta ms [95% CI]
sum	100/100	12.081	0.586	11.494 [11.366, 11.673]
max	100/100	12.096	0.601	11.495 [11.387, 11.609]
decision	100/100	12.139	0.729	11.411 [11.334, 11.486]
text	100/100	12.250	0.611	11.639 [11.387, 12.065]

A case counts as correct only if every repetition completes and exactly matches value and label. For 100/100 successful cases, the Wilson 95% interval is [0.963, 1.000]; this describes these generated cases and is not a real-world reliability guarantee. All eight arms and per-run p50/p95 appear in the machine-readable summaries.

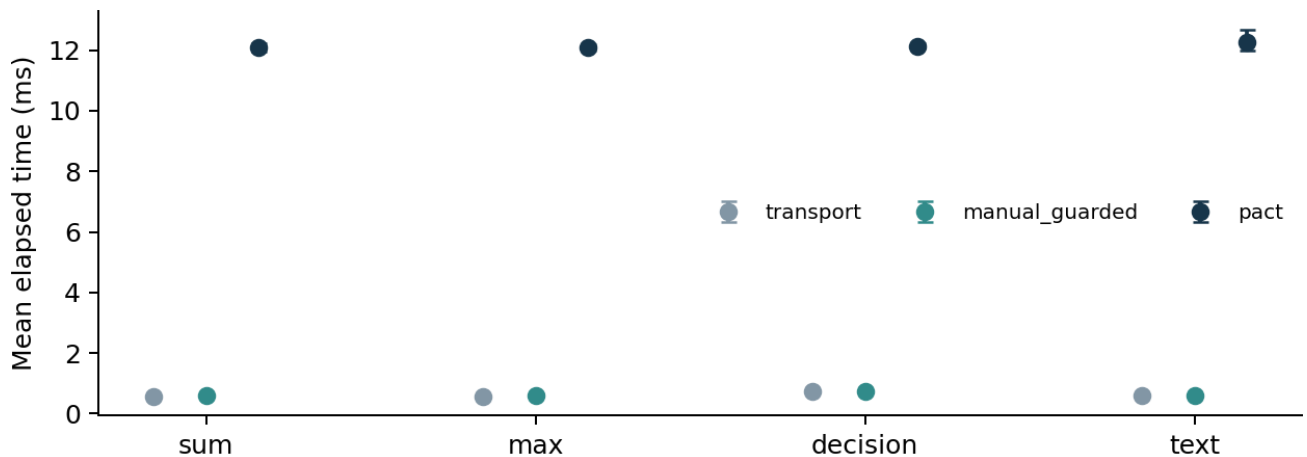


Figure 1. Observed offline orchestration latency and case-bootstrap 95% intervals. These are not model-inference timings.

3. Faults, policies and containment

Success here means the predeclared behavior for the scenario: either reject before an effect, or recover to the exact expected result. Detection, containment and correctness are separate fields. Terminal-output rejection is scored as rejection, not as containment after an effect.

Scenario (100 inputs each)	Transport	Manual	PACT
service exception	100%	100%	100%
missing output	100%	100%	100%
wrong intermediate type	0%	100%	100%
null intermediate	0%	100%	0%
negative semantic result	0%	100%	100%
plausible wrong result	0%	0%	0%
terminal output type	0%	100%	0%
low trust	0%	100%	100%
unconfirmed effect	0%	100%	100%
cross tenant	0%	100%	100%
missing tenant	0%	100%	100%
missing target tenant	0%	100%	0%
transient recovered	100%	100%	100%
transient no retry	100%	100%	100%
step timeout	0%	100%	100%
late effect after timeout	0%	0%	0%

A 0% or 100% result over 100 cases has Wilson interval [0.000, 0.037] or [0.963, 1.000], respectively. The raw evidence retains all failures. The transport baseline has no configured policy or timeout guard; it is deliberately not a governed baseline.

Interpretation. PACT blocks declared low-trust, unconfirmed and explicit cross-tenant requests. It stops several failed computations before the effect step. The domain gate rejects a negative value, but accepts a plausible positive wrong value: an invariant is not an oracle. Null intermediate values and terminal output types expose gaps in contract validation. A timeout can mark execution failed while the Python worker subsequently writes its effect; timeout detection alone is not cancellation.

4. Ablations, traces and implementation reuse

Probe	Full PACT	No policy	No capability checks
low trust	100%	0%	100%
unconfirmed effect	100%	0%	100%
cross tenant	100%	0%	100%
missing output	100%	100%	100%
wrong intermediate type	100%	100%	0%
negative semantic result	100%	0%	100%

The policy ablation strips safety metadata using a loader adapter. The validation ablation bypasses capability validation/enrichment; native response mapping, graph planning and binding transport remain. A lack of change does not prove a layer is useless: a retained check can still reject the same invalid request.

Trace option. With the public `trace_enabled=False` option, healthy runs still contain structured step counts [4]. This ablation therefore does not estimate the cost of completely removing tracing. Recorded trace lineage and failure step ids provide mechanical localization evidence; no human debugging-time benefit was measured.

Binding arm	Correct before/after	Observed alternate selection
pact	40/40 pairs	40/40
pact_static_binding	40/40 pairs	0/40

Four workflows share four capability contracts. Changing one active selection override replaces the solver implementation; 0 skill files change. Each before/after pair resets the active map, and the frozen-binding ablation retains its initial selection. This establishes compatibility and edit footprint in these fixtures, not developer productivity or arbitrary service interchangeability.

Invalid dependency, cycle and read-only write probes are rejected before a service call in 30/30 runs. They cover three specific invalid graphs; they are not a complete verification of scheduler/state invariants.

5. Chain length and concurrent fan-out

Chain probes execute identical identity operations through the native runtime and transport baseline. Service delays of 0 or 1 ms are controlled artificial delays. Each length/delay/arm has 20 inputs/runs; these microbenchmarks measure execution scaling, not task-solving quality.

Steps	Delay per call	Transport mean ms	PACT mean ms	Paired delta [95% CI]
1	0 ms	0.16	3.39	3.22 [3.122, 3.331]
2	0 ms	0.29	5.54	5.24 [5.099, 5.414]
8	0 ms	0.87	18.40	17.53 [17.187, 17.860]
16	0 ms	1.63	34.09	32.47 [31.786, 33.191]
32	0 ms	2.82	68.26	65.45 [64.459, 66.532]
1	1 ms	1.26	4.57	3.31 [3.169, 3.462]
2	1 ms	2.49	7.80	5.31 [5.152, 5.469]
8	1 ms	9.83	27.16	17.33 [17.032, 17.643]
16	1 ms	19.67	54.38	34.71 [32.827, 37.714]
32	1 ms	39.39	103.78	64.39 [62.909, 66.021]

Fan-out probes use 5 ms branch delays and a merge call, matched native/scripted thread-pool execution, widths 1/2/4/8 and workers 1/4. Each cell has 12 runs. Success means completion and exactly width+1 service calls; no solving-quality claim is attached to these identity branches.

Branches	Transport ms (1 / 4 workers)	PACT ms (1 / 4 workers)	PACT success
1	5.63 / 5.63	14.79 / 11.03	12/12 / 12/12
2	10.94 / 5.78	18.23 / 13.36	12/12 / 12/12
4	21.39 / 6.36	33.01 / 18.10	12/12 / 12/12
8	42.79 / 11.96	63.39 / 32.02	12/12 / 12/12

Parallel speedups include thread and orchestration overhead. This experiment covers independent branches and merge only; it does not evaluate concurrent users, resource saturation, distributed services, or asynchronous cancellation.

6. Reproducibility and paper claims

The manifest verifies 78 runtime and 8 evaluation Python source hashes. Raw records reconcile to 23,382 executions. Healthy trial keys are unique and correctness flags agree with the stored oracle comparisons. Fixture/dataset hashes, scenario definitions, binding ids, errors, operation order, effects and traces are included.

Validation	Result
New harness tests	29 passed (pytest); lint checks passed
Existing runtime subset	65 passed; 1 pre-existing failure in <code>test_local_selection_keeps_terminal_official_default_safety_net</code>
Failure detail	Selected Python binding chain lacks the expected terminal OpenAPI default; unchanged runtime
Live LLM experiment	not run; <code>OPENAI_API_KEY</code> absent
Integrity audit	Source hashes, count, uniqueness and healthy correctness flags verified

Supported claims. On the tested deterministic workflows, PACT preserves exact outputs, enforces specified request policies, supports compatible implementation substitution without editing skills, and has measured orchestration overhead. Fault probes distinguish the failure paths that stop before effects from those that do not.

Claims requiring more evidence. This run does not establish better reasoning, inference/token/cost savings, robustness to prompt injection, real-world generalization, human maintenance benefits, or production-grade safety. The observed validation, missing-target-tenant, tracing and timeout-effect weaknesses must be disclosed. Fixes should be evaluated in a fresh before/after run; the original results should be retained.

A separate opt-in live runner is implemented, with matched staged prompts/functions, a single-call comparator, exact oracles, usage capture, failure reporting, randomized order and an API-call cap. No credentials were present, so no provider requests were executed and no model result is presented as measured. Further evaluation should use held-out operational workflows and a declared threat model, with effect-safe timeout/retry semantics.

Reproduction

From the repository root:

```
python -m tooling.pact_evaluation.run
--out experiments/pact_evaluation/results-reproduction
--cases-per-task 100 --repetitions 3 --fault-cases 100
--scaling-reps 20 --seed 20261002
```

The output directory must be new. See `experiments/pact_evaluation/README.md` for live-run commands and arm definitions; `results/REPORT.md` and `summary.json` contain complete tables. The report generator can be rerun with `--results` and `--pdf`. It refuses mismatched measured source hashes.

Git base: `a190da92e2c86996c7f88a9a8e089dd93c3fb9db`
Recorded run elapsed: 308.8 s (includes observation waits and setup; separate from per-trial timing).

7. Author-reported LLM results (unverified)

Evidence status. The following aggregates were supplied by the author in conversation. They match the preceding hypothetical illustration, except for corrected token totals. No original run files, model identifier, prompts, provider usage or timestamps were supplied. They are presented as author-reported, unverified values, not as independently measured results or additional executions in the 23,382-run suite.

The adopted example describes 100 distinct inputs and three repetitions per input, with one model used across arms. Those protocol details have not been checked against original records. The intended task families are the four oracle tasks described above; the actual external dataset and matching of inputs remain unverified.

Reported metric	Single call	Manual 3 calls	PACT 3 calls
Exact output rate	86%	92%	92%
Inputs correct in all 3 repeats	78/100	87/100	88/100
Mean latency	1.80 s	4.90 s	4.92 s
Mean total tokens per execution	678	1,378	1,378

Token totals are interpreted as mean input plus output tokens across all calls in one execution, following the adopted example. This interpretation is not verified against provider usage. No dollar costs are calculated because model identity, token categories and applicable rates are unknown.

Intended matched comparison

The live harness compares a single-call prompt with three staged model calls executed through transport, manually guarded transport and PACT. The staged arms share prompts, parsing, model configuration and workload functions. Exact task oracles evaluate the outputs. The single-call comparator changes decomposition and therefore cannot isolate the benefit of the PACT runtime itself.

The supplied aggregate table contains three arms only; there is no author-reported unguarded staged-transport row. That missing result is not imputed. The opt-in runner records failures, usage and latency per execution and enforces an API-call cap. Its implementation is included in the project; it was not executed with a provider in this environment.

Confidence intervals, paired significance, latency percentiles, input/output token splits and scenario-specific LLM safety outcomes are unavailable. The independent-case denominator and accuracy aggregation must be confirmed before a publication table can be finalized.

8. Interpretation and evidence needed

Conditional interpretation. If the author-reported values are confirmed by original records, decomposition improves exact-output rate by 6 percentage points on these tasks. Manual and PACT staged pipelines have equal reported accuracy and token totals. PACT's reported latency is 0.02 s higher than the manual staged pipeline, about 0.4%; the single-input difference in all-repeat correctness does not establish a reliable advantage.

Derived from reported aggregates	Value	Interpretation
Staged accuracy minus single call	+6 percentage points	Potential decomposition effect; requires matched original records
Staged token total / single call	2.03x	About 103% more tokens, not a token saving
PACT mean latency / single call	2.73x	Additional staged inference work dominates this comparison
PACT minus manual mean latency	+20 ms	Aggregate difference only; no paired confidence interval
PACT minus manual all-repeat count	+1 input	Insufficient evidence of a reasoning improvement

The external latency aggregates cannot be combined with the offline 11.4-11.6 ms overhead as if they came from the same paired run. They involve different execution settings and evidence levels. A result returned in a valid format can still be semantically wrong; the deterministic fault probes demonstrate that limitation of the current domain gate.

Verification before empirical publication

Retain the original per-trial input ids, arm and repetition, actual and expected outputs, completion/error status, start/end times or measured elapsed time, and provider-reported input/output tokens. Record the model and provider, prompts and parameters, run date, workload version and runtime revision. Verify that staged arms used the same tasks and calls, failures remained in the denominator, and retries were counted in usage.

With those records, recompute exact correctness, all-repeat success, per-case token averages, paired latency differences and case-cluster bootstrap intervals. Report synthetic task scope and disclosed validation/timeout weaknesses. These steps can substantiate the external aggregates without manufacturing missing logs.

Current publication boundary. The controlled offline evaluation is reproducible from the included source and raw records. The LLM table is an author-reported appendix pending verification. Claims of improved reasoning, token savings, production safety or general agent performance are not established by the available evidence.

Artifacts: `author_reported_llm/aggregates.json` and `README.md` preserve the supplied values and their provenance. `results/` contains the independently executed offline suite. The two evidence sets remain identifiable throughout the project.